Lab 6: Debugging with `jdb` & Exception Handling

Breakpoints · Checked vs. unchecked · `try/catch` · Custom exceptions

---

Alex (Aizierjiang Aiersilan)
Lab Lead, Teaching Assistant

October 5, 2026

The George Washington University
CSCI 2113 - Software Engineering | Fall 2026

© Aizierjiang Aiersilan. Free distribution and open-access.

Outline

1. Lab 6 at a Glance
2. Debugging with jdb
3. Exceptions from Library Methods
4. Custom Exceptions
5. The Assignment
6. Wrap-up
7. Acknowledgement

ng Aiersilan | CSCI 2113

Before we begin: check your midterm submission access

Do this first — on the lab laptop

Before starting today's lab, we need to verify that **each student** can access the course submission server from the lab laptop that will be used for the midterm.

1. On the **lab laptop**, open:

   `https://submit.cs.seas.gwu.edu/courses`

2. **Log in using your own account.** Access is individual, so every student must verify their own login.
3. Confirm that you can reach the course submission page successfully.

Why are we checking this now?

The lab laptops are heavily restricted. We want to catch any access or login problems **before the midterm**, rather than spending exam time troubleshooting them.

Test a submission from the lab laptop

1. Create a small **dummy file** on the lab laptop. A blank or simple text file is sufficient.
2. From the appropriate test/practice submission entry on the submission server, **upload and submit the dummy file**.
3. Verify that the website accepts the submission from the lab laptop.

Important

The goal is **not** to work on or submit the actual midterm today. This is only a connectivity and submission test to make sure the lab laptop can access the submission server and successfully upload a file.

If something does not work

Tell the instructor/TA **during today's section** so the access issue can be addressed before the midterm.

Lab 6 at a Glance

ng Aiersilan | CSCI 2113 | F

What you will learn today

Four goals

1. Use **jdb** to find a bug that gives wrong output without crashing.
2. Read the **API documentation** to find what a method throws, and tell **checked** from **unchecked**.
3. Handle exceptions with `try`/`catch`.
4. Define, throw, and catch a **custom exception**.

Three activity parts + assignment

→ §1 Debugging with **jdb** `practice/`
→ §2 Library exceptions `practice/`
→ §3 Custom exceptions `lab6/`
→ Assignment: invalid input + *complete a ride*

Due

Tuesday, **October 13, 2026**, **11:59 PM**

Setting up

```
lab6/
|-- practice/          # Sections 1 and 2
|   |-- Driver.java
|   |-- Leaderboard.java
|   |-- fares.txt
|   `-- FareReport.java
|-- (all Lab 5 assignment files)
`-- UserNotFoundException.java   # Sec. 3
```

- §1–§2 live in `practice/`; §3 and the assignment live in `lab6/`.
- The code uses `IO.println`: class **java.lang.IO** is new in **Java 25** [7, 3].
- Check first: `java -version` should report **25** or later.

Tip

Copy the handout code *exactly*: the debugger refers to **line numbers**.

ng Aiersilan | CSCI 2113

Debugging with jdb

ng Aiersilan | CSCI 2113 | F

A bug that does not crash: `Leaderboard.java`

```
1  public class Leaderboard {
2      static void sortByRating(Driver[] drivers) {
3          for (int i = 0; i < drivers.length - 1; i++) {
4              int best = i;
5              for (int j = i + 1; j < drivers.length; j++) {
6                  if (drivers[j].getRating() > drivers[i].getRating()) {
7                      best = j;
8                  }
9              }
10             Driver temp = drivers[i];
11             drivers[i] = drivers[best];
12             drivers[best] = temp;
13         }
14     }
16     public static void main(String[] args) {
17         Driver[] drivers = {
18             new Driver("Ada", 4.2),
19             new Driver("Bob", 4.9),
20             new Driver("Cy", 4.5),
21             new Driver("Dee", 3.8),
22             new Driver("Eve", 4.7)
23         };
24         sortByRating(drivers);
25         for (Driver d : drivers) {
26             IO.println(d);
27         }
28     }
29 }
```

Selection sort: pass `i` finds the index `best` of the highest rating in `drivers[i..length-1]`; lines **10–12** swap it into position `i`.

Symptom: it runs, but the ranking is wrong

Actual output

```
Eve 4.7   ⚠
Bob 4.9   ⚠
Cy 4.5
Ada 4.2
Dee 3.8
```

Expected

```
Bob 4.9
Eve 4.7
Cy 4.5
Ada 4.2
Dee 3.8
```

Try a small input

Rank only Ada (4.2), Cy (4.5), Bob (4.9):
output is *correct*.

⚠ A passing small test does not prove the code correct.

No exception, no stack trace, nothing to point at. → We need to **watch the program's state** while it runs. Restore the five drivers.

Step 0: compile with -g, then start jdb

```
$ javac -g *.java
$ jdb Leaderboard
Initializing jdb ...
>
```

- -g = **all debugging info**, including **local variables** [8].
- Default `javac`: only line numbers + source file name.
- **jdb** *loads* the program but does not run it until you type `run` [9].

Forgot -g? (real **jdb** output)

```
main[1] print best
com.sun.tools.example.debug.expr
  .ParseException: Name unknown: best
 best = null
main[1] locals
Local variable information not
available.  Compile with -g to
generate variable information
```

⚠ Breakpoints still work (line info is there); variable names do not.

Session 1: localize — is the inner loop's answer right?

Predict *before* you look

After pass **0**, `best` should be Bob's index: **1**.

- `stop at Leaderboard:10` — first line after the inner loop.
- The program pauses *before* line **10** runs.
- `where` prints the **call stack**.
- `print` on an object shows its `toString()`.

```
> stop at Leaderboard:10
> run
Breakpoint hit: "thread=main",
  Leaderboard.sortByRating(), line=10 bci=46
10                Driver temp = drivers[i];

main[1] where
  [1] Leaderboard.sortByRating (Leaderboard.java:10)
  [2] Leaderboard.main (Leaderboard.java:24)
main[1] print best
 best = 4
main[1] print drivers[best]
 drivers[best] = "Eve 4.7"
```



Prediction **1**, reality **4** (Eve) → the bug is inside the inner loop. Type `exit`.

Session 2: watch every comparison on line 6

```
> stop at Leaderboard:6
> run
main[1] locals
Method arguments:
drivers = instance of Driver[5] (id=432)
Local variables:
i = 0
best = 0
j = 1
main[1] print drivers[j]
 drivers[j] = "Bob 4.9"
main[1] print drivers[best]
 drivers[best] = "Ada 4.2"
main[1] cont
```

Each stop is *before* line **6** runs:

i	j	best	drivers[j]	drivers[best]
0	1	0	Bob 4.9	Ada 4.2
0	2	1	Cy 4.5	Bob 4.9
0	3	**2**	Dee 3.8	**Cy 4.5**
0	4	2	Eve 4.7	Cy 4.5

The suspicious stop

At `j=3`, `best` has *dropped* from Bob (**4.9**) to Cy (**4.5**).

So at `i=0`, `j=2` the test on line **6** said Cy beats the best so far. Why?

The object `id` (**432** here) can differ from run to run.

Root cause and fix

```
// line 6, buggy: compares with the FIRST unsorted driver
if (drivers[j].getRating() > drivers[i].getRating()) {
// line 6, fixed: compares with the best seen SO FAR
if (drivers[j].getRating() > drivers[best].getRating()) {
```

At `i=0, j=2`: Cy **4.5** > Ada **4.2** ✔ ... but Cy **4.5** < Bob **4.9**.

The buggy loop picks the last driver who beats `drivers[i]`, not the best.

After the fix

```
$ javac -g *.java
$ java Leaderboard
Bob 4.9
Eve 4.7
Cy 4.5
Ada 4.2
Dee 3.8
```

💡 Why Ada, Cy, Bob passed: in ascending input the *last* driver who beats `drivers[i]` is also the best one.

jdb cheat sheet

Command	Effect
`stop at Class:line`	set a breakpoint at a line
`clear Class:line`	remove a breakpoint
`run`	start the program
`cont`	continue to the next breakpoint
`where`	print the call stack
`locals`	print arguments and local variables
`print expr`	evaluate an expression, print result
`next`	run the line, *stepping over* calls
`step`	run the line, *stepping into* calls
`exit`	leave **jdb**

The method, not just the tool

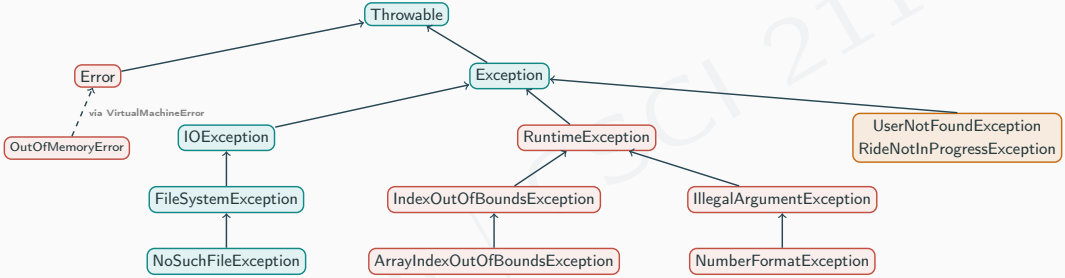
1. **Predict** a value.
2. **Stop** where it should hold.
3. **Compare**; narrow the region.
4. Re-read that line; fix; re-run.

Scientific debugging [10]; commands per **jdb** docs [9].

Exceptions from Library Methods

ng Aiersilan | CSCI 2113 | F

The exception hierarchy: checked vs. unchecked



■ **Checked**: `Throwable` and subclasses *except* `RuntimeException`, `Error` and theirs [2]. The compiler forces you to handle them.

■ Unchecked: `RuntimeException`, `Error`, and subclasses. Usually a *bug*. ■ **Yours** (this lab): checked.

Read the API docs: `Files.readAllLines(Path)`

```
public static List<String> readAllLines(Path path) throws IOException
```

Throws section [6]

- `IOException` — if an I/O error occurs reading from the file or a malformed or unmappable byte sequence is read

`IOException` is **checked** → the compiler will hold every caller to it.

Compare: `Double.parseDouble`

```
public static double parseDouble(String s)
    throws NumberFormatException
```

Declared too — but unchecked, so the compiler does *not* enforce it.

💡 Habit: click the exception class and read the **superclass chain** at the top of its page — `IOException` extends `Exception`, *not* `RuntimeException`.

The compiler insists: catch it or declare it

```
1  import java.nio.file.Files;
2  import java.nio.file.Path;
3  import java.util.List;
4
5  public class FareReport {
6      public static void main(String[] args) {
7          List<String> lines = Files.readAllLines(Path.of(args[0]));
8          IO.println(lines.size() + " fares");
9      }
10 }
```

```
FareReport.java:7: error: unreported exception IOException;
must be caught or declared to be thrown
```

```
// add: import java.io.IOException;
try {
    List<String> lines =
        Files.readAllLines(Path.of(args[0]));
    IO.println(lines.size() + " fares");
} catch (IOException e) {
    IO.println("Could not read file: " + e);
}
```

The **catch-or-specify** requirement [4]: code that can throw a **checked** exception must `catch` it or declare it with `throws`.

A `catch` matches the type *and every subclass*

```
$ java FareReport fares.txt
3 fares
$ java FareReport missing.txt
Could not read file: java.nio.file.NoSuchFileException: missing.txt
```

The object caught is a `NoSuchFileException`:

`NoSuchFileException` → `FileSystemException` → `IOException`

so `catch (IOException e)` catches it.

`"..." + e` vs. `e.getMessage()`

`+ e` calls `toString()`: class name + message

`e.getMessage()`: just `missing.txt`

Order matters: most specific `catch` first

✗ Superclass first

```
} catch (IOException e) {
    IO.println("Could not read file: " + e);
} catch (NoSuchFileException e) {
    IO.println("No such file: " + e.getMessage());
}
```

```
error: exception NoSuchFileException has
already been caught
```

✓ Subclass first

```
} catch (NoSuchFileException e) {
    IO.println("No such file: " + e.getMessage());
} catch (IOException e) {
    IO.println("Could not read file: " + e);
}
```

```
$ java FareReport missing.txt
No such file: missing.txt
```

The JVM runs the first matching `catch`; a later block that can never run is a **compile-time error** [2].

Unchecked = usually a bug: *prevent* it, don't catch it

```
$ java FareReport
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException:
Index 0 out of bounds for length 0
    at FareReport.main(FareReport.java:...)
```

`ArrayIndexOutOfBoundsException` → `IndexOutOfBoundsException` → `RuntimeException`: the compiler never asked.

✔ Check the precondition

```
if (args.length == 0) {
    IO.println("Usage: java FareReport <file>");
    return;
}
```

Guideline: recoverable → **checked**; programming error → unchecked [5, 1].

`Double.parseDouble` and reading a stack trace

```
double total = 0;
for (int i = 0; i < lines.size(); i++) {
    total += Double.parseDouble(lines.get(i));
}
IO.println(lines.size() + " fares, total "
    + String.format("$%.2f", total));
```

```
3 fares, total $70.25
```

`NumberFormatException` →

`IllegalArgumentException` →

`RuntimeException`: unchecked.

Change line 3 of `fares.txt` to `22.7S`:

```
Exception in thread "main"
java.lang.NumberFormatException:
For input string: "22.7S"
  at java.base/...FloatingDecimal.check(...)
  at java.base/...    (2 more JDK frames)
  at java.base/java.lang.Double
       .parseDouble(Double.java:...)
  at FareReport.main(FareReport.java:17)
```

💡 Read from the top for *what*; scan down to the first line in *your* file for *where*. (Your line number depends on your layout.)

Where you put the `try` decides what keeps running

Catch outside the loop

- Prints `Not a fare: ...`
- Fares after the bad line are ✗ not read; no total.
- After a `catch` finishes, control does not return into the `try`.

Catch inside the loop

```
double total = 0;
int count = 0;   // valid fares only
for (int i = 0; i < lines.size(); i++) {
    try {
        total += Double.parseDouble(lines.get(i));
        count++;
    } catch (NumberFormatException e) {
        IO.println("Line " + (i + 1)
            + " is not a fare: " + lines.get(i));
    }
}
IO.println(count + " fares, total "
    + String.format("$%.2f", total));
```

```
Line 3 is not a fare: 22.7S
2 fares, total $47.50
```

Custom Exceptions

ng Aiersilan | CSCI 2113 | F

Why a custom exception? The trouble with `null`

In Lab 5, `RideShare.findDriver(id)` returns `null` when no driver has the id.

- ✗ Every caller must remember to check.
- ✗ The compiler does not enforce it.
- ✗ Forget once → `NullPointerException` later, far away.

Lab 6 move

`findDriver` throws a **checked** **UserNotFoundException**.

✓ Now the compiler makes every caller deal with "not found".

ng Aiersilan | CSCI 2113

Define it, throw it, declare it

```
public class UserNotFoundException extends Exception {
    public UserNotFoundException(String message) {
        super(message);   // stored; getMessage()
    }
}
```

```
// in RideShare.findDriver: replace  return null;
throw new UserNotFoundException("No driver with id " + id);
```

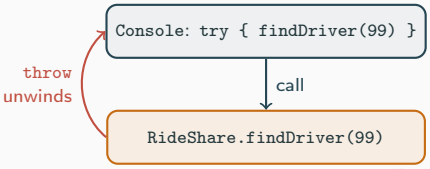
```
error: unreported exception UserNotFoundException;
must be caught or declared to be thrown
```

`throw`	a *statement*: raise an exception object now
`throws`	part of a *declaration*: "this method may raise. . ."

```
public Driver findDriver(int id)
        throws UserNotFoundException
```

Extends `Exception` → **checked**. The constructor hands the message to `Exception`'s constructor.

Declaring passes the obligation to the caller




```
// Console (sketch): remove the null check
try {
    Driver d = rideShare.findDriver(id);
    // ... toggle d and print, as in Lab 5
} catch (UserNotFoundException e) {
    IO.println(e.getMessage());
}
```

```
Your choice: 5
Driver id: 99
No driver with id 99
```

Same output as Lab 5 — but a caller can no longer forget the check.

Same shape for riders: `findRider`

Step 1 — lookup lives in `RideShare`, Lab 5 style:

```
public Rider findRider(int id)    // returns null
```

If your lookup is written inside `Console`, *move* it into `findRider`.

Step 2 — convert it like `findDriver`:

```
public Rider findRider(int id)
        throws UserNotFoundException
```

`Console` calls it inside `try/catch`:

```
Your choice: 7
Rider id: 99
No rider with id 99
```

One lookup, one place

Refactor first, then change behavior: two small, safe steps.

ng Aiersman / CSU 2113

The Assignment

ng Aiersilan | CSCI 2113 | F

Assignment: from “crashes on bad input” to “handles every input”

§1 Handling invalid input

- **Numbers**: re-ask until an integer
- **Choices**: menu, ride type, y/n
- **Rule-breaking values**: `IllegalArgumentException`
- **Unknown ids**: **UserNotFoundException**; an unknown *rider* id returns to the menu *before* the rest of the ride is asked

§2 Completing a ride

- New menu option **(10)**
- `RideShare.completeRide(int rideId)`
- New checked **RideNotInProgressException**
- Completed rides list with | `COMPLETED`

Starts from the end of the Activity (`findDriver`/`findRider` already throw). No `practice/` needed.

Numbers: one method that asks until it gets an integer

```java
// Pattern sketch (name and input call are yours)
static int promptInt(String prompt) {
    while (true) {
        String input = IO.readln(prompt);
        if (input == null) {          // end of input: stop,
            System.exit(0);           // or it loops forever
        }
        try {
            return Integer.parseInt(input);
        } catch (NumberFormatException e) {
            IO.println("Not a number: " + input);
        }
    }
}
```

```
Zone: three
Not a number: three
Zone: 3
```

Use it for every number prompt: `Zone`, `Seats`, `Driver id`, `Rider id`, `Pickup zone`, `Dropoff zone`, `Party size`, `Ride id`.

Watch out

Print the *input*, not `e.getMessage()` (that would say `For input string: "three"`). `IO.readln` returns `null` at end of input, and `parseInt(null)` throws too: without the `null` check the loop prints `Not a number: null` forever.

Choices: three prompts, three behaviors

Prompt	Valid	On bad input print	Then
Main menu	a listed option (1–10, q)	`Invalid choice:` + input	print the menu again
Ride type	1, 2, or 3	`Invalid ride type:` + input	ask again, without reprinting types
Luxury (y/n)	`y` or `n`	`Please answer y or n`	ask again

⚠ Lab 5 never rejected another ride type (the handout says it was treated as Standard). Lab 6 must ✖ reject it.

💡 Ride type and the menu are *not* number prompts: compare the **text**, so `abc` reports `Invalid ride type: abc`.

Values that break a rule: the owner throws `IllegalArgumentException`

Value	Rule	Thrown by	Message
Driver's zone	0 to `zoneCount() - 1`	`RideShare.signUpDriver`	`Invalid zone: 7`
Seats	at least **1**	`Vehicle` constructor	`Invalid seat count: 0`
Pickup / dropoff zone	0 to `zoneCount() - 1`	`Ride` constructor	`Invalid zone: 7`
Party size	at least **1**	`Ride` constructor	`Invalid party size: 0`

```
if (seats < 1) {
    throw new IllegalArgumentException(
        "Invalid seat count: " + seats);
}
```

- Applies to *every* `Vehicle` and `Ride` constructor.
- `Vehicle` no longer silently turns a seat count below **1** into **1**.
- Several rules broken? Any check order; report only the first failure.
- `Console` catches it, prints the message, returns to the menu.

“Nothing is created”: check first, then commit

✗ Commit first

```
int id = nextId++;           // id used up!
Vehicle v = new Vehicle(..., seats, ...);
// throws -> id is gone forever
```

✓ Validate / construct first

```
// 1. check zone (may throw)
// 2. new Vehicle(...)   (may throw)
// 3. only now: take an id, add to lists
```

```
Your choice: 1
Name: Grace
Zone: 2
Plate: DC-2002
Seats: 0
Luxury (y/n): y
Invalid seat count: 0
```

A failed sign-up adds no driver and uses no id; a failed request books no ride and adds nothing to the waitlist. **Failure atomicity** [1].

⚠ Field names above are illustrative — apply the idea to *your* Lab 5 design.

§2 Completing a ride (menu option 10)

Completing a ride in progress:

1. marks the ride **completed**;
2. moves its driver to the ride's **dropoff zone**;
3. makes the driver **available**;
4. **serves the waitlist** with that driver, exactly as a Lab 5 toggle does.

No ride in progress? Print `No rides in progress` and return.

```
Your choice: 10
Ride #1 (Luxury): Ana with Grace | 14 miles | $40.00
Ride #2 (Standard): Alan with Ada | 7 miles | $11.25
Ride #3 (XL): Ana with Linus | 11 miles | $22.75
Ride id: 2
Completed Ride #2: Ada is now AVAILABLE in zone 4
Booked Ride #4 (Standard): Alan with Ada | 5 miles | $8.75
```

First it lists the rides *in progress* in the format of option 8.

RideNotInProgressException and listing rides

Checked; thrown by `completeRide` when *no ride in progress* has the id (never booked, or already completed).

```
Ride id: 9
Ride #9 is not in progress
```

A completed ride in option 8 (Ride #2 from the previous slide):

```
Ride #2 (Standard): Alan with Ada | 7 miles | $11.25 | COMPLETED
```

New required methods

```
RideShare:
public Ride completeRide(int rideId)
  throws RideNotInProgressException
Driver: public void setZone(int zone)
Ride: public void complete()
      public boolean isCompleted()
class RideNotInProgressException
  extends Exception: constructor
public RideNotInProgressException(
  String message)
```

Rules that will be checked

- ✗ Do not catch `Exception`, `RuntimeException`, or `Throwable` — catch the *specific* types in the handout.
- ✗ Do not catch bug signals (`ArrayIndexOutOfBoundsException`, `IndexOutOfBoundsException`, `NullPointerException`) — *prevent* them.
- ✗ No empty `catch` blocks.
- ✓ No exception escapes for any input that ends with `q`.
- ✗ No class from `java.util`.
- ✗ No type-name checks or `instanceof` in `Ride` or its subclasses — keep **overriding**.
- ✓ Helper methods/constructors are fine; all `.java` in one directory.

💡 "Don't ignore exceptions" [1]: an empty `catch` hides the very bug you need to see.

ng Aiersilan | CSCI2113

Check, package, submit

```
# 1. run the checker (jar next to your .java files)
$ java -jar check-lab6.jar

# 2. package
$ tar -czf lab6-assignment.tgz *.java              # Linux
$ tar --no-mac-metadata -czf lab6-assignment.tgz *.java  # Mac
PS> tar -czf lab6-assignment.tgz *.java            # Windows
```

3. Upload

submit.cs.seas.gwu.edu
→ CSCI 2113: Software-Engineering
→ Lab 6 - Assignment
→ upload `lab6-assignment.tgz`

Deadline

Tue, **Oct 13, 2026**, **11:59 PM**

Aiersilan

Wrap-up

ng Aiersilan | CSCI 2113 | F

Takeaways

1. ★ **Debug by prediction**: state the value you expect, stop with **jdb**, compare, narrow.
2. ★ **Checked** exceptions are part of a method's contract (`throws`); unchecked ones usually mean a bug to *prevent*.
3. ★ `catch` matches subclasses; put the most specific first; *where* you catch decides what keeps running.
4. ★ A **custom checked exception** turns "remember to check for `null`" into a rule the **compiler enforces**.

In one sentence

Find bugs by comparing expectation with state, and handle failures where the program can actually recover.

Acknowledgement

ng Aiersilan | CSCI 2113 | F

Acknowledgement

This lab builds on the initial materials developed by **Timothy Kim**.

We gratefully acknowledge his contribution.

ng Aiersilan | CSCI 2113

References i



J. Bloch.
Effective Java.
Addison-Wesley, third edition, 2018.
Chapter 10, Items 69–77 (Exceptions).



J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, and G. Bierman.
The Java Language Specification, Java SE 25 Edition.
Oracle America, Inc., 2025.
Chapter 11 (Exceptions) and §14.20 (The try statement).
`https://docs.oracle.com/javase/specs/jls/se25/html/jls-11.html`.



OpenJDK.
JEP 512: Compact source files and instance main methods.
OpenJDK. `https://openjdk.org/jeps/512`, 2025.
Delivered in JDK 25.

References ii



Oracle.

The catch or specify requirement.

The Java Tutorials, Essential Java Classes: Exceptions. https://docs.oracle.com/javase/tutorial/essential/exceptions/catchOrDeclare.html.

Accessed Oct. 4, 2026.



Oracle.

Unchecked exceptions — the controversy.

The Java Tutorials, Essential Java Classes: Exceptions. https://docs.oracle.com/javase/tutorial/essential/exceptions/runtime.html.

Accessed Oct. 4, 2026.



Oracle.

Class Files (java.nio.file), method readAllLines(Path).

Java SE 25 & JDK 25 API Specification. https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/nio/file/Files.html, 2025.

Accessed Oct. 4, 2026.

References iii



Oracle.

Class IO (java.lang).

Java SE 25 & JDK 25 API Specification. `https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/IO.html`, 2025.

Since Java 25. Accessed Oct. 4, 2026.



Oracle.

The javac command.

Java SE 25 & JDK 25 Tool Specifications.

`https://docs.oracle.com/en/java/javase/25/docs/specs/man/javac.html`, 2025.

Option -g. Accessed Oct. 4, 2026.

ng Aiersilan / CSCI 2113



Oracle.
The jdb command.
Java SE 25 & JDK 25 Tool Specifications.
`https://docs.oracle.com/en/java/javase/25/docs/specs/man/jdb.html`, 2025.
Accessed Oct. 4, 2026.



A. Zeller.
***Why Programs Fail: A Guide to Systematic Debugging*.**
Morgan Kaufmann, second edition, 2009.

Thank you!

Questions?

Backup: other `try` forms (not required in Lab 6)

`finally`

```
try {
    work();
} catch (IOException e) {
    report(e);
} finally {
    cleanUp(); // runs either way
}
```

Multi-catch

```
try {
    work(); // may throw
            // NoSuchFileException
} catch (NoSuchFileException
       | NumberFormatException e) {
    report(e);
}
```

Alternatives may not be subclasses of each other.

try-with-resources

```
try (var in = Files
        .newBufferedReader(p)) {
    use(in);
} // in.close() is automatic
// IOException: still catch/declare
```

All three are specified in JLS §14.20 [2].

Backup: questions students often ask

- **`next` vs. `step`?** `next` runs the line and stays in this method; `step` goes *into* a called method (e.g. `getRating()`).
- **Why does `locals` at line 10 not show `j`?** `j` is declared in the inner `for`; at line **10** it is out of scope.
- **Catching `IllegalArgumentException` in `Console`?** It also matches `NumberFormatException` (a subclass). Keep number parsing inside your integer-prompt method so it never escapes.
- **Is `" 3"` a number?** `Integer.parseInt(" 3")` throws: it does not trim spaces (`Double.parseDouble` does).

ng Aiersilan / CSCI 2113